SOFTWARE REQUIREMENTS SPECIFICATION

Portfolio Management System (MERN)

The Portfolio Management System is a MERN-based portfolio platform with a public showcase and a protected admin panel for managing profile, projects, skills, certifications, and contact messages.

APPLICATION TYPE	Personal Portfolio + Admin CMS
ARCHITECTURE	React + Express + MongoDB + JWT
DEPLOYMENT	Render Static Site + Render Web Service + MongoDB Atlas

1. Purpose and Scope

The purpose of this system is to provide an online portfolio for showcasing professional details and work, while also providing a secure admin interface to manage all portfolio content without editing code directly.

Core Objectives

- Provide a responsive public portfolio website with smooth section-based navigation.
- Allow authenticated admin users to manage profile, projects, skills, and certifications.
- Capture visitor messages through a contact form and show them in admin dashboard.
- Store all application data in MongoDB with REST APIs for structured access.
- Support deployment-ready setup for frontend and backend services.

2. Functional Overview

- **User-Facing:** Home, About, Skills, Projects, Academic Results, Certifications, Contact.
- **Admin-Facing:** Login, dashboard summary, CRUD management pages, message center.
- **Content Types:** Profile data, project entries, skill entries, certificate entries, contact messages.
- **System Behavior:** Public read APIs, protected write APIs, JWT-based authorization for admin actions.

3. Phase 1: System Design

Primary Data Models

Model	Key Fields
User	email, passwordHash, role
Profile	name, headline, summary, photoUrl, links
Project	title, description, techStack, keyFeatures, outcome, featured, urls
Skill	name, category, level, order
Certificate	title, issuer, issueDate, imageUrl, description
Message	name, email, subject, message, createdAt, isRead

System Flow

- Visitor opens Home page and navigates through sections.
- Public pages fetch content through REST APIs.
- Visitor can submit contact message through contact form.
- Admin logs in with JWT and manages all content modules.
- Changes reflect immediately on public pages after save.

4. Phase 2: Environment Setup

Required Tools

- Node.js and npm
- VS Code
- MongoDB Atlas account
- Git + GitHub repository
- Postman (API testing)
- Render account (frontend + backend deployment)

Expected Setup

- Client (`client`) and server (`server`) directories configured.
- Environment variables configured for both frontend and backend.
- MongoDB Atlas connectivity verified.
- API routes tested successfully.
- Root script starts frontend and backend concurrently for local development.

5. Phase 3: Backend (Core APIs)

Module	Endpoints	Purpose
Auth	/api/auth/login, /api/auth/me	Admin authentication and token-based session validation.
Projects	/api/projects (GET/POST/PUT/DELETE)	Create and manage portfolio projects.
Skills	/api/skills (GET/POST/PUT/DELETE)	Manage categorized skill records.
Certificates	/api/certificates (GET/POST/PUT/DELETE)	Manage certification records and media links.
Profile	/api/profile (GET/PUT)	Read and update public profile details.
Messages	/api/messages (POST/GET/DELETE)	Store visitor contact submissions and admin message handling.

6. Phase 4: Authentication (JWT)

JWT is used to secure admin-only APIs. After successful login, the token is stored on the client side and passed in protected API requests. Backend middleware validates token authenticity before allowing CRUD access on protected resources.

- Token issued after login.
- Protected routes guarded by auth middleware.
- Unauthorized requests receive 401/403 responses.

7. Phase 5: Frontend Setup

Frontend is built with React and Vite. Routing is handled by React Router. Axios is used for API communication with Render-hosted backend.

Public Pages

- HomePage.jsx
- AboutPage.jsx
- SkillsPage.jsx
- ProjectsPage.jsx
- AcademicResultsPage.jsx
- CertificationsPage.jsx
- ContactPage.jsx

Admin Pages

- AdminLoginPage.jsx
- AdminDashboardPage.jsx
- ManageProfilePage.jsx
- ManageProjectsPage.jsx
- ManageSkillsPage.jsx
- ManageCertificatesPage.jsx
- ManageMessagesPage.jsx

8. Phase 6: Data Seeding and Restore Utilities

- Seed scripts are included to restore key content quickly.
- Root command `npm run seed:all` restores profile, skills, certificates, and projects.
- Additional scripts support project and skill upserts for curated portfolio data.

9. Phase 7: Contact and Communication Module

The Contact module allows visitors to submit inquiries with name, email, subject, and message. Submissions are stored in MongoDB and visible to the admin panel for follow-up and cleanup.

- Public contact form validation.
- Message persistence in database.
- Admin-side message review and deletion.
- User-friendly success and error notifications.

10. Phase 8: Deployment

Layer	Platform	Role
Frontend	Render Static Site	Hosts React portfolio UI with SPA rewrite.
Backend	Render Web Service	Runs Express API and authentication flows.
Database	MongoDB Atlas	Stores users, profile, projects, skills, certificates, messages.

Live URLs

- Frontend: https://portfolio-web-zco6.onrender.com/
- Backend: https://portfolio-lpbd.onrender.com/
- Health Check: https://portfolio-lpbd.onrender.com/api/health

11. Non-Functional Requirements

- **Performance:** Responsive UI and efficient API responses for standard portfolio traffic.
- **Security:** JWT-protected admin routes, environment-based secrets, and credential rotation support.
- **Usability:** Clean public UI, clear navigation, and mobile responsiveness.
- **Maintainability:** Modular frontend/backend structure and separate controller/model/routes architecture.
- **Scalability:** CRUD-first API design supports future extensions (blogs, testimonials, analytics, etc.).

12. Conclusion

This SRS defines the complete scope, architecture, modules, and deployment strategy of the Portfolio Management System. The platform is production-deployed and supports both public portfolio presentation and secure admin content management through a scalable MERN architecture.

Prepared as Software Requirements Specification for Portfolio Management System. Generated on 18/04/2026, 00:14.